

UNIVERSITY OF NIŠ
FACULTY OF ELECTRONIC ENGINEERING



Vedran Mitić 2123

SEMINAR PAPER

Data Quality Engineering for Training Generative Decoder-Only Transformers

Supervisor:
Prof. Dr. Aleksandar Stanimirović

2026

Table of Contents

1. Introduction.....	3
2. Decoder-Only Architecture and Data Dependency	4
2.1. Operating Principle: Next-Token Prediction and the Influence of Data Structure...4	
2.2. Pre-training (Quantity) vs. Fine-tuning (Quality).....	5
2.3. Current Models and Their Data Sources.....	5
3. Dimensions of Text Data Quality	7
3.1. Lexical Parameters.....	7
3.2. Noise in Large-Scale Data	8
3.3. Data Contamination	9
3.4. Algorithmic Overlap Detection: MinHash and LSH	10
4. Preprocessing and Normalization Techniques.....	11
4.1. Modern Tokenizers	11
4.2. Deduplication.....	11
4.2.1. Repetition Loop	11
4.2.2. Memorization and Generalization.....	12
4.2.3. Jaccard Similarity Index	12
4.3. Language Identification	12
5. Data Curation and Alignment	14
5.1. Heuristic Filtering: Linguistic Rules and Safety.....	14
5.2. Model-Based Filtering	14
5.3. SFT and RLHF.....	16
6. The Role of Semantic Models in Modern Preprocessing	17
6.1. LLM-as-a-Judge.....	17
7. Evolution Toward Synthetic Preprocessing: Knowledge Generation and Distillation..	18
7.1. Data Distillation and Teacher-Student Dynamics.....	18
7.2. Self-Correction and Automated Reasoning	18
7.3. Curriculum Learning.....	18
7.4. Risk of Model Collapse	18
8. Conclusion	20
References.....	21

1. Introduction

In the modern era of artificial intelligence, the dominance of **Decoder-only** transformer architectures — led by models from the *GPT* and *Llama* families — has fundamentally transformed the approach to natural language processing (NLP). While early research efforts were primarily focused on increasing the number of parameters and computational power, the latest trends in the development of **large language models (LLMs)** unambiguously indicate that data quality has become a more critical factor than model size itself. Findings on the optimal relationship between model size and data volume (*scaling laws*) have shown that the availability of high-quality data is becoming the primary bottleneck to further progress.

This paper analyzes the complex transformation pipeline from raw, noisy internet text to high-quality corpora suitable for training generative models. Focusing on the specifics of Decoder-only architectures, the research encompasses classic quality concepts such as lexical diversity and noise, while extending them to modern challenges such as document-level deduplication, semantic filtering, and subword tokenization. Through theoretical analysis, this paper aims to identify how adequate preprocessing directly contributes to reducing hallucinations and increasing the factual accuracy of models, laying the groundwork for the development of more robust and reliable artificial intelligence systems.

This data dependency takes on particular significance in light of the *Garbage In, Garbage Out (GIGO)* principle, which in the domain of generative transformers acquires new, more complex implications. Unlike traditional machine learning models — where the impact of low-quality data was primarily reflected in reduced classification accuracy — the consequences for Decoder-only architectures are far-reaching and harder to quantify. Since these models operate on the principle of autoregressive next-token probability prediction, they inherently internalize all statistical anomalies, logical inconsistencies, and factual errors present in the input corpus. Consequently, the presence of noise and factual errors in the data can increase the probability of generating inaccurate or hallucinated responses, as the model optimizes a probability distribution without an explicit truth-verification mechanism. In this sense, preprocessing and filtering represent the key mechanism for controlling the distribution over which the model is optimized, thereby directly influencing the stability and reliability of the generated outputs.

The primary objective of this paper is a systematic analysis of the evolution of data preprocessing techniques and their role in optimizing the performance of Decoder-only transformer models. The research seeks to deconstruct the transition from traditional, static text-cleaning methods — based on heuristics and regular expressions — to modern, dynamic approaches involving AI-assisted data curation. The paper will examine in detail how specific quality parameters, such as lexical density and structural coherence, affect the model's ability to generate semantically correct content. In addition to the theoretical framework, the paper aims to identify optimal strategies for mitigating the negative effects within large-scale corpora, providing a scientifically grounded insight into how rigorous preprocessing serves as the primary defense against factual degradation and systematic bias in generative systems.

2. Decoder-Only Architecture and Data Dependency

Understanding the critical role of data quality requires a prior analysis of the architectural specifics of the models that process such data. Modern large-scale language models are primarily based on the Decoder-only variant of the transformer architecture, which distances itself from the original encoder-decoder design by focusing exclusively on the generative aspect of information processing. While earlier architectures aimed at mapping an input sequence into a fixed vector space for classification or translation, Decoder-only systems are designed for modeling the probability distribution of language through *causal self-attention*. This architectural decision implies that the model learns not only semantic relationships, but also the structure itself, noise, and statistical distortions present in the corpus. Accordingly, model architecture and data quality cannot be viewed in isolation; they form a symbiotic system where the structural integrity of the input information directly determines the cognitive capacity and precision of the final model.

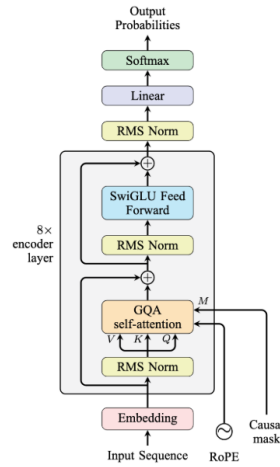


Figure 1. Architecture of a Decoder-only Transformer [1]

2.1. Operating Principle: Next-Token Prediction and the Influence of Data Structure

The essence of training Decoder-only transformers lies in the language modeling task of *next-token prediction*. For a given token sequence $S = (x_1, x_2, \dots, x_n)$, the model seeks to maximize the log-probability of each subsequent element based on all preceding ones, formally expressed through the probability function:

$$P(S) = \prod_{i=1}^n P(x_i | x_1, \dots, x_{i-1}; \theta) \quad (1)$$

where θ represents the network parameters optimized during the learning process. This autoregressive approach implies that data structure plays a decisive role in shaping the internal representations of the model. When the input data exhibits high coherence and proper syntactic structure, the causal self-attention mechanism successfully identifies long-range dependencies within the text. However, when the input sequence is contaminated by disruptions in logical flow — such as improperly removed HTML tags or fragmented sentences due to aggressive document truncation — the model is forced to model statistical noise instead of linguistic rules. Such structural data degradation directly leads to increased uncertainty in the model's predictions and slower convergence during optimization.

The specificity of the Decoder-only architecture lies in the application of the *causal masking* mechanism within the self-attention layers. This mechanism ensures that when predicting a token at position i , the model can only access information from positions $j < i$, thereby simulating a strictly linear flow of time. However, this architectural restriction makes the model extremely sensitive to the quality of the preceding context. If the initial segment of the sequence is saturated with noise, orthographic errors, or semantic gibberish, the attention mechanism will incorporate such representations into contextual projections, which propagate to higher network layers through residual connections and multilayer transformations. In a process known as *autoregressive error amplification*, the model is forced to build future predictions on unstable foundations, resulting in a cumulative decline in coherence during text generation. Thus, the causal mask not only defines the direction of learning, but also serves as a transmission channel through which poor input data quality is directly projected onto the final model output.

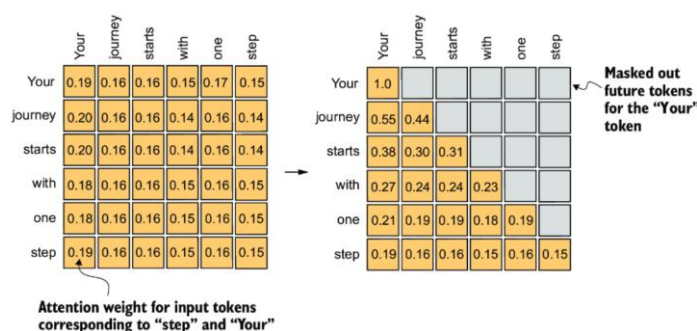


Figure 2. Causal Mask [2]

2.2. Pre-training (Quantity) vs. Fine-tuning (Quality)

The difference in data requirements across the lifecycle of a transformer is best illustrated by the distinction between the pre-training and fine-tuning phases. During pre-training, the primary goal is to expose the model to the broadest possible spectrum of linguistic phenomena, where data quantity plays a decisive role in forming basic world knowledge and linguistic competence. In contrast, the fine-tuning phase (and its variations such as instruction tuning) requires a drastically smaller, but high-quality and strictly curated dataset, used to steer the model toward specific tasks or a desired communication tone. While the first phase tolerates a certain degree of variation in quality in order to achieve statistical robustness across trillions of tokens, the second phase is extremely sensitive to any deviation, as even a small number of inadequate examples can disrupt previously acquired knowledge or introduce undesirable behavioral patterns.

2.3. Current Models and Their Data Sources

The evolution of modern systems — from the pioneering versions of the GPT series to the latest iterations such as Llama 3 and GPT-4 — is inextricably linked to the development of massive digital text repositories. While early models used relatively homogeneous datasets, such as BookCorpus or the Wikipedia corpus, modern systems rely on heterogeneous sources that cover a significant portion of the entire public internet. The key source for most of these models is *Common Crawl* — a vast database of indexed web pages that provides the necessary quantity, but simultaneously represents the greatest challenge in terms of data cleanliness. To overcome the extremely high level of noise in raw web content, specialized corpora have been developed, such as *C4 (Colossal Clean Crawled Corpus)*, which uses rigorous heuristics to remove redundant and low-quality fragments, and *The Pile*, which aims for maximum diversity by combining academic publications,

source code, and legal documents. An analysis of these sources reveals that the success of a model depends not only on the raw volume of text (measured in terabytes), but on the sophistication of the algorithms that perform primary data selection before the training process itself.

3. Dimensions of Text Data Quality

Defining quality in the context of unstructured textual information presents a complex challenge that goes beyond a simple binary classification into correct and incorrect data. Corpus quality in modern machine learning is viewed through a multidimensional framework encompassing statistical properties, technical cleanliness, and the semantic integrity of content. While early preprocessing approaches were primarily focused on removing obvious noise, the development of complex generative systems has imposed the need for a deeper analysis of lexical parameters and document structure. Understanding these dimensions is crucial, as every anomaly in the input data — whether it be statistical monotony, the presence of technical artifacts, or contamination by test examples — directly affects the model's ability to generalize knowledge and maintain a high degree of creativity without losing factual accuracy.

3.1. Lexical Parameters

Statistical analysis of the lexical composition of a corpus provides an initial insight into its informational value. One of the basic quality indicators is **document length**, which must be in line with the model's intended context window. Excessively short documents often do not contain enough information to establish complex semantic connections, while extremely long texts without a clear structure can burden the attention mechanisms with unnecessary redundancies.

Beyond length, **vocabulary size** and the **lexical diversity index** represent key metrics for avoiding the phenomenon of linguistic impoverishment. Lexical diversity is often quantified using the *Type-Token Ratio (TTR)* metric, which represents the ratio of unique words to the total number of words in the text:

$$TTR = \frac{V}{N} \quad (2)$$

where V denotes the number of unique lexemes and N is the total number of tokens in the sample. A high value of this index indicates a rich and diverse corpus, which is a prerequisite for the model to develop a broad range of expressive capabilities. Conversely, low diversity — often caused by repetitive phrases or machine-generated spam — can cause the model to become statistically trapped in a narrow subset of language, resulting in monotonous and predictable text generation.

One of the most reliable indicators of the naturalness and quality of a corpus is its compliance with **Zipf's Law**. This empirical law states that the frequency of any word in a language is inversely proportional to its rank in the frequency table, formally expressed as:

$$P_r = \frac{C}{r^\alpha} \quad (3)$$

where P_r is the probability of occurrence of the word of rank r , C is a constant, and the exponent α is usually close to 1. In the context of preprocessing for large language models, significant deviations from this distribution within a document often indicate low-quality data. For example, excessive representation of certain keywords — characteristic of texts optimized for search engines (SEO spam) — or an extremely small number of unique lexemes in long sequences signal that the data lacks natural linguistic structure. Such documents are often rejected in the filtering phase, as they do not contribute to generalization and can cause the model to become prone to aimless phrase repetition.

In practical application, the data selection process relies on strictly defined **heuristic thresholds** that serve as initial quality filters. Research teams (such as those who worked on GPT-3 and Llama models) apply a set of rules based on statistical extremes. For example, documents are automatically eliminated if they have:

- **Extreme word lengths:** An average word length that significantly deviates from the average for a given language (e.g., over 15 characters in English) often indicates the presence of source code, chemical formulas, or random character strings.
- **Low stop-word ratio:** If a document contains too few function words (such as prepositions and conjunctions), it is likely a product list, tabular data, or generated spam, rather than fluent text.
- **High symbol ratio:** Documents in which punctuation marks, emojis, or numerical symbols dominate over alphabetic characters are considered noise that can degrade the learning process.

An additional metric that addresses the limitations of raw TTR is unigram entropy, borrowed from information theory. While TTR measures the fraction of unique words, it does not account for how evenly those words are distributed throughout the document. Unigram entropy resolves this by measuring the unpredictability of the word distribution, computed as:

$$H = -\sum (\text{count}(w) / N) * \log(\text{count}(w) / N)$$

where the sum is taken over all unique words w in the document and N is the total number of words. A document where one word dominates will have low entropy regardless of its TTR, making this metric a more reliable indicator of vocabulary diversity. In practice, high-quality natural language corpora typically exhibit entropy values between 5.0 and 7.0, while values below 3.0 indicate linguistic impoverishment — repetitive phrasing, machine-generated content, or structural data fragments rather than fluent text.

3.2. Noise in Large-Scale Data

The presence of noise in corpora intended for training Decoder-only transformers is an unavoidable consequence of using automated data collection methods such as web scraping. Noise can be classified into two basic categories:

- **Technical artifacts:** These encompass residual HTML tags, metadata, and JavaScript fragments which, if not adequately removed, force the model to expend valuable parameters on learning markup language syntax instead of natural language. The presence of residual HTML elements — such as `<div>` tags or non-breaking space characters (` `) — represents a significant burden for the tokenization process. Modern models use a limited context window (e.g., 8,192 or 32,768 tokens). When a document contains unpurified HTML code, each symbol and tag is converted into a separate token, consuming precious memory space that should be reserved for semantically relevant text. Such redundancy directly reduces the model's efficiency, as it expends computational resources on learning the markup syntax of web pages instead of understanding the logical relationships within natural language. A particular challenge is posed by *OCR errors* (Optical Character Recognition) arising from the digitization of printed materials, where incorrectly recognized characters create non-existent lexemes and increase vocabulary entropy. A typical example is the visual similarity of characters, where the lowercase letter

'l' is recognized as the number '1', or the letter 'o' as zero '0'. In the sentence "The level of quality is high", an OCR error can generate the string "The 1eve1 Of qua1ity is high". For the model, the word "1eve1" represents a completely new, unknown token with no semantic connection to the original word "level". Such errors fragment the model's vocabulary, increase entropy, and prevent proper generalization, as the model treats these typographic errors as legitimate but unrelated linguistic entities.

- **Semantic degradation:** This manifests through *boilerplate text*, which includes repetitive elements such as navigation menus, legal notices in page footers, and aggressive advertising content. These elements not only introduce unnecessary redundancy, but can distort the model's attention, weakening its ability to establish coherent connections within the primary content of a document. The main problem with such data is its high frequency of occurrence combined with minimal informational value. If a million pages in the training set each contain an identical footer with terms of service text, the model will statistically favor that sequence, which can lead to overfitting on administrative vocabulary. Additionally, boilerplate text often interrupts the natural flow of sentences; for example, a paragraph may be interrupted by a navigation menu fragment such as "Home | Products | Contact", which confuses the attention mechanism and makes it difficult to learn long-range dependencies within the text. Filtering such content is essential to ensure that the model learns from fluent, coherent human expression rather than the technical structure of web presentations.

3.3. Data Contamination

One of the most critical challenges regarding corpus quality for modern Decoder-only models is **data contamination**. This phenomenon involves the unintentional inclusion of examples from publicly available test sets (benchmark sets) in the model training corpus. Given that models such as GPT-4 or Llama are trained on massive extractions from the global web, there is a high probability that the data collection process will also capture pages from GitHub repositories, academic portals, or platforms such as Hugging Face, where questions and solutions for standard intelligence and knowledge benchmarks are stored (e.g., MMLU, GSM8K, or HumanEval).

The problem of contamination directly undermines the model evaluation process; instead of demonstrating generalization ability and logical reasoning, the model may simply recite memorized answers it has seen during the training phase. Such artificially inflated accuracy creates a false impression of the system's cognitive capabilities, which in the scientific community raises justified doubts about the actual performance of LLMs. Identifying and removing contaminated data requires the application of advanced preprocessing methods, such as n-gram overlap between the training corpus and test sets. If a high degree of overlap in long token sequences is detected, it is necessary to filter those documents to ensure that the evaluation results reflect the model's true intelligence rather than mere statistical memorization of solutions.

An analysis of technical reports from state-of-the-art models confirms the seriousness of this problem. The authors of *GPT-3* acknowledged in their original paper a filtering error where, due to a code bug, a portion of test sets "leaked" into the training corpus. Although they attempted to retroactively analyze the impact of this contamination, they acknowledged that it may have led to unjustifiably high results on certain benchmark tests.

Similarly, in the technical report for *Llama 2*, the Meta research team devoted an entire chapter to contamination analysis. They conducted rigorous cleaning using n-gram overlap

(where $n = 13$), eliminating any document that had more than 13 consecutive identical tokens with any test example. Notably, even after such cleaning, models showed traces of memorization, suggesting that traditional filtering methods may be insufficient to prevent models from rote-learning widely available internet benchmarks. These examples underscore that combating contamination is an ongoing process requiring constant innovation in preprocessing techniques. [3][4]

3.4. Algorithmic Overlap Detection: MinHash and LSH

Detecting contamination on datasets spanning trillions of tokens requires the application of extremely efficient algorithms for approximate string comparison. In this context, the **MinHash** algorithm represents the standard for estimating the Jaccard similarity index between two documents without requiring direct word-by-word comparison. The process involves transforming documents into sets of n -grams, which are then mapped through hash functions into a fixed array of minimum hash values (signatures). The similarity between two documents is estimated based on the probability that their MinHash values are identical.

To further accelerate this process, the **Locality-Sensitive Hashing (LSH)** technique is used. LSH groups similar MinHash signatures into common buckets, thereby drastically narrowing the search space. Instead of comparing every document in the enormous training set with every question in the test set (an $O(n^2)$ operation), LSH allows comparisons to be performed only within the same buckets. This methodology is of vital importance for preserving model integrity, as it enables researchers to identify and eliminate in real time not only documents that contain identical copies of test questions, but also their close variations or paraphrased versions.

4. Preprocessing and Normalization Techniques

Advanced preprocessing and normalization techniques are not merely optional cleaning steps, but form the very core of the algorithmic pipeline that defines how the model will interpret linguistic units. At this stage, the focus is on resolving problems of redundancy, linguistic uniformity, and representational efficiency through sophisticated tokenization and deduplication methods. The implementation of these techniques directly addresses the previously mentioned GIGO problem, ensuring that the model expends its computational resources on learning distinctive semantic patterns rather than on processing statistically insignificant repetitions or inadequately encoded characters.

4.1. Modern Tokenizers

The tokenization process represents the first level of abstraction at which a text string is transformed into numerical vectors intelligible to a neural network. While older models used whole-word or character-level tokenization, modern systems rely on **subword** methods, primarily *Byte-Pair Encoding (BPE)* and *SentencePiece*.

BPE operates through the iterative merging of the most frequent pairs of adjacent characters into new, longer tokens. This approach allows the model to balance efficiency (for common words) and flexibility (by decomposing rare or unknown words into meaningful morphemes). On the other hand, *SentencePiece* introduces an additional innovation by treating whitespace as a symbol, which enables uniform processing regardless of language-specific characteristics, avoiding the need for pre-tokenization based on language-specific rules.

The quality of the tokenization algorithm directly affects model performance in domains requiring precision, such as mathematics and programming:

- **Number interpretation:** If a tokenizer splits multi-digit numbers into inconsistent fragments (e.g., the number "1024" is seen as "10" and "24", while "1025" is seen as "1", "0", and "25"), the model will have significant difficulties learning arithmetic operations and numerical relations.
- **Source code comprehension:** In languages like Python, where indentation is critical to code logic, poor tokenization that ignores or incorrectly merges whitespace can render code completely non-functional for the model.

4.2. Deduplication

Deduplication of documents in corpora intended for generative models is divided into two levels: **exact** and **fuzzy** deduplication. Exact deduplication removes identical copies of documents that were created through multiple indexing of the same pages by web crawlers. However, a far greater challenge is fuzzy deduplication, which identifies documents with minimal variations (e.g., the same news article published on different portals with different headlines). As previously mentioned in the context of contamination, algorithms such as MinHash and LSH find their primary application here, enabling semantic similarity comparisons at the scale of trillions of examples.

4.2.1. Repetition Loop

The key reason for rigorous deduplication lies in preventing the phenomenon where the model becomes trapped in infinite repetitive loops during inference. Research has shown that if a particular sequence or writing style appears excessively in the training set, the

model assigns disproportionately high probability to those sequences. The result is a model that, after generating a few sentences, falls into a loop of repeating the same phrases, because through redundant data it has learned that the statistically most likely continuation of text is precisely what it just wrote.

4.2.2. Memorization and Generalization

Beyond generation stability, deduplication directly affects the ability to generalize. Models trained on duplicated content tend toward **memorization** (overfitting) rather than understanding logical concepts. By removing duplicates, the model is forced to learn universal linguistic patterns from more diverse sources, which significantly reduces the risk of verbatim reproduction (plagiarism) of training set segments. In modern pipelines — such as the one used for Llama 3 — deduplication is performed at multiple levels, from the sentence level to the full document level, effectively reducing the token count by up to 30%, while simultaneously dramatically increasing the informational density of the remaining data.

4.2.3. Jaccard Similarity Index

The **Jaccard similarity index** is used to compare the n-gram sets of two documents (A and B). Mathematically, this index represents the ratio of the intersection to the union of token sets:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} \quad (4)$$

In practical application on large corpora, researchers most commonly set thresholds between **0.8 and 0.9**. Documents whose Jaccard index exceeds this value are considered redundant, and only one representative copy is retained. Such granularity allows the system to recognize and eliminate variations of the same content (e.g., articles differing only in date or city name), thereby directly preventing an excessive concentration of certain statistical pathways within the neural network.

Beyond improving generation quality, deduplication also has a critical economic justification. Training models with hundreds of billions of parameters requires thousands of GPU-hours, where every unnecessarily processed trillion tokens costs millions of dollars in electricity and computing resources. Effective deduplication can reduce the raw dataset by **20% to 40%** without degrading model performance. In the scientific literature, this phenomenon is often described as an increase in the informational density of the corpus [6]; the model achieves the same or higher level of competence (benchmark results) in a shorter training period, because the learning process focuses exclusively on unique informational units. In this way, rigorous preprocessing becomes a key factor in the democratization of LLM development, enabling top-tier results with optimized hardware resource utilization.

4.3. Language Identification

The process of **Language Identification (LID)** represents the primary filter in corpus construction, tasked with accurately classifying documents according to the language in which they are written. In the era of multilingual models, this step is crucial for controlling the distribution of knowledge; if a significant percentage of documents in a related or dominant language (e.g., English) appear in the training set for a specific language (e.g., Serbian), the model may develop a tendency toward code-switching or syntactic

contamination. Modern systems use fast classifiers based on character n-grams or lightweight neural networks, such as the FastText model, which achieve high precision even on short text fragments. Documents for which the classifier cannot determine the language with high confidence (usually above 0.9) are most often discarded, as they are assumed to contain excessive noise, source code, or are simply meaningless character strings.

Following language identification, the normalization phase aims to reduce text to a standardized form without loss of semantic information. Normalization in the LLM context encompasses several critical operations:

- **Unicode character unification:** Resolving issues with different code pages and normalizing visually identical symbols with different digital representations, thereby preventing unnecessary vocabulary expansion.
- **Punctuation and whitespace treatment:** Standardizing spaces, removing control characters, and unifying different types of quotation marks and dashes, which directly affects tokenization consistency.
- **HTML entity decoding:** Transforming symbols such as & or < into their textual equivalents (& and <), ensuring the model learns from natural symbols rather than their web-encoded substitutes.

Proper language identification and normalization ensure that the model builds robust vector representations from clean, uniform data. Without these steps, the model would be forced to expend its capacity on learning variations in the encoding of the same linguistic symbols, resulting in reduced efficiency and poorer generalization for low-resource languages where every correct token is of critical importance.

Special focus in the preprocessing process must be placed on **low-resource languages**, where the amount of available digital data is significantly smaller compared to global languages such as English or Chinese. For such languages, each document carries greater relative weight in shaping the model's internal representations. Therefore, excessively aggressive filtering and strict language identification thresholds can be counterproductive, as they may eliminate rare but high-quality information sources. The normalization strategy for these languages often requires finer tuning; for example, for Serbian, it is necessary to ensure proper mapping between the Cyrillic and Latin scripts so that the model does not treat the same language as two completely different entities, which would effectively halve the available learning capacity.

Additional complexity in language identification is introduced by the presence of dialects, localisms, and informal internet language. Standard LID algorithms (such as FastText) are often trained on formal texts (Wikipedia, news agencies), which can cause them to misclassify dialects or archaic forms as noise or as a different, related language. For example, texts in the Torlak dialect or specific regional expressions may be rejected as unknown or incorrect. In modern preprocessing, this problem is addressed by introducing more inclusive classifiers that recognize variations within the same language code. The goal is to preserve linguistic diversity, as these data enable the model to develop a more natural communication tone and a better understanding of specific cultural contexts that standard corpora often do not contain.

5. Data Curation and Alignment

Once the raw data has undergone technical normalization and deduplication, the critical phase of **curation** begins, transforming quantity into usable value. Data curation represents the process of selective filtering and categorization of information with the goal of exposing the model only to those examples that promote the development of desired cognitive abilities. While the early training phase is focused on absorbing as much information as possible, the **alignment** phase serves as the final level of preprocessing that ensures the model's output is not only accurate, but also safe, useful, and aligned with human values. This process combines automated heuristics, AI-based classifiers, and direct human feedback, making the boundary between raw data and intelligent response clear and controlled.

5.1. Heuristic Filtering: Linguistic Rules and Safety

The first level of curation relies on a set of strictly defined **heuristics** that serve for the mass elimination of documents that, despite being technically correct, have no educational or informational value. These heuristics are most commonly based on statistical ratios within the text:

- **Word-to-number ratio:** Documents dominated by numerical data without accompanying textual explanations (such as large tables or log files) are often rejected, as they do not contribute to learning linguistic structures.
- **Presence of toxic content:** The application of block lists of words (profanity, hate speech, explicit content) serves as a primary barrier. However, modern preprocessing must be careful not to eliminate important historical, literary, or medical documents that may contain sensitive terms in a scientific context. Such sanitization not only limits the model's general knowledge, but can render it incapable of recognizing and adequately responding to inappropriate content in real interactions. Contemporary research suggests that instead of complete deletion, it is more effective to use data weighting, where controversial but informative content is retained with special metadata tags that guide the model toward an objective tone.
- **Sentence quality:** Heuristics verify whether sentences begin with a capital letter and end with a punctuation mark. Documents with too many grammatically incorrect fragments are flagged as low quality.

In practice, most research teams today follow strict protocols established during the development of models such as **Gopher** (DeepMind) and **C4** (Google). The Gopher protocol introduces a set of rules based on statistical thresholds, where, for example, any document is rejected if the ratio of symbols to words exceeds a certain limit, or if the proportion of stop-words (e.g., "and", "the", "of") is too low, which typically signals that the text was generated robotically. The C4 (Colossal Clean Crawled Corpus) standard focuses on eliminating repetitive elements ("lorem ipsum" texts) and filtering based on sentence length. These standards have become the foundation of modern curation, as they enable the derivation of a high-quality database that retains linguistic diversity while minimizing noise — from hundreds of terabytes of raw data from Common Crawl.

5.2. Model-Based Filtering

When the capabilities of heuristic filtering are exhausted, the phase of **model-based filtering** begins, where smaller, specialized machine learning models are used to make qualitative judgments about text. While heuristics focus on surface structure, these models

(such as fastText classifiers or specific variants of BERT) are trained to recognize the semantic value of a document. The main goal is binary or graded classification of content into *High-quality* (informative, coherent text) and *Low-quality* (spam, machine-generated content, fragmented thoughts). This process most often proceeds in two steps:

- 1. Training a classifier on a gold standard:** Researchers manually curate a smaller dataset (e.g., Wikipedia articles, selected scientific papers, and quality books) that serve as positive examples. Randomly selected, unfiltered portions of web crawl corpora are used as negative examples.
- 2. Mass inference:** The auxiliary model (classifier) then passes through billions of documents, assigning each a quality score. Documents whose utility probability falls below a certain threshold (e.g., $P < 0.65$) are automatically eliminated from the final training set.

The advantage of this approach lies in its ability to recognize subtle nuances that the human eye easily perceives but are difficult to define in code. For example, the model can successfully identify text that is grammatically correct but informationally empty or repetitive (gibberish). The use of models such as fastText is particularly popular here due to its extreme speed; it can process hundreds of thousands of tokens per second, making it economically viable for filtering corpora measured in terabytes.

In addition to classifiers, one of the most effective techniques for filtering large corpora is measuring the *perplexity* of a document using a smaller, already-trained model. Perplexity is a measure of how surprising a given text is for the model; mathematically, it represents the exponent of cross-entropy:

$$PP(W) = P(w_1, w_2, \dots, w_N)^{-\frac{1}{N}} \quad (5)$$

where $P(w_1, w_2, \dots, w_n)$ is the probability of the entire text sequence W , and N is the length of the text. A low PP value (e.g., $< 2-5$) indicates text that is too predictable — often sequences like "here comes text here comes text" or automatically generated server logs. The model learns nothing new from this, only reinforcing superfluous, basic patterns. A high PP value (e.g., $> 100-200$, depending on the model) indicates chaotic text — often randomly assembled words for SEO optimization, texts in an unrecognized language, or documents with so many OCR errors that they have lost semantic meaning. In preprocessing practice, documents with extremely low or extremely high perplexity are most often rejected. By setting optimal perplexity thresholds, researchers ensure that the training set contains texts that are sufficiently predictable to be comprehensible, yet sufficiently informative to promote learning of complex dependencies.

The introduction of models as quality judges carries the risk of systemic bias. Since these classifiers are trained on data labeled by humans as high-quality (such as encyclopedic articles or legal documents), they tend to favor exclusively formal and academic writing styles. This can lead to the unintentional elimination of informal but valuable sources such as forums, social media dialogues, or creative writing — which are crucial for learning colloquial speech and slang. If the classifier becomes too rigid, the model trained on such a purified corpus may become capable of writing scientific papers but completely unsuccessful at understanding everyday human communication or specific cultural nuances. Resolving this problem requires careful balancing of the training set for the classifiers themselves, to ensure that high quality encompasses the full spectrum of human expression, not just strictly formal discourse.

5.3. SFT and RLHF

Although large-scale preprocessing and automated filtering remove technical noise, they cannot guarantee that the model will be useful, safe, or aligned with human intent. For this reason, the final stage of data preparation introduces ***Supervised Fine-Tuning (SFT)*** and ***Reinforcement Learning from Human Feedback (RLHF)***. These methods represent a specific form of curation where the emphasis shifts from quantity to extreme quality and precision of information.

SFT relies on high-quality datasets created or verified by humans (usually domain experts). These data consist of instruction-response pairs (e.g., "Explain matrix multiplication to a 5-year-old" — followed by an ideal answer). In this phase, preprocessing focuses on eliminating any ambiguity; each response must be factually accurate, stylistically consistent, and free of bias. SFT serves as the gold standard that teaches the model the basics of dialogue and task resolution.

RLHF represents an even more advanced level of processing, where humans do not write direct responses but rather rank model outputs. The process proceeds through several steps:

1. The model generates several different responses to the same query.
2. Human evaluators rank those responses (e.g., from best to worst) based on criteria such as truthfulness, usefulness, and safety.
3. These rankings are used to train a Reward Model, which then serves as an automatic critic that penalizes bad and rewards good responses during further learning.

This phase is crucial for resolving problems of toxicity and safety that automated filters could not fully eliminate. Human feedback enables the model to understand subtle contexts, irony, or potentially dangerous queries that are grammatically correct but ethically unacceptable. In this way, RLHF and SFT act as the ultimate filters that transform the model from a passive information repository into a proactive and safe system ready for interaction with users.

6. The Role of Semantic Models in Modern Preprocessing

While previous phases focused on removing noise and obviously bad data, modern preprocessing introduces the concept of **semantic filtering**. The goal is no longer merely to have "clean" text, but text that possesses high **educational value**. Here, vector spaces and semantic embeddings are used to map the entire knowledge within a corpus, enabling researchers to balance topics and identify the most informative parts of the internet. [9][10]

Instead of looking at individual words, modern systems use embedding models to convert documents into vectors in a high-dimensional space. Using clustering algorithms (such as k-means), researchers can identify thematic groups within trillions of documents. This is crucial for:

- **Identifying rare domains:** Finding high-quality texts on quantum physics or rare programming languages that would otherwise be overshadowed in a sea of celebrity news.
- **Balancing diversity:** If 90% of the corpus consists of news, the model will be biased. Semantic clustering enables engineers to intentionally sample data so that every topic (science, art, code, law) is represented in an optimal ratio, regardless of how frequently it appears on the raw internet.

The similarity between two documents in this space is most commonly computed using **cosine similarity**:

$$\cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|} \quad (6)$$

where A and B are vector representations of documents. The closer the value is to 1, the more semantically related the documents are, which enables even more precise deduplication than MinHash.

6.1. LLM-as-a-Judge

In the pursuit of the perfect training set, researchers have introduced a methodology where a highly developed model (such as GPT-4 or Claude 3) serves as a judge that evaluates the quality of raw data for training smaller or newer models. This process goes beyond binary filtering and introduces a graded quality scale. The judge model receives an instruction to rate a document on a scale of 1 to 10 based on specific criteria: **logical coherence, informational density, and absence of semantic noise**. In this way, the preprocessing process acquires a new dimension — qualitative ranking. Documents that receive low scores (e.g., below 3) are permanently discarded, while those with the highest scores (above 8) are repeated multiple times in the training set (up-sampling). Such stratification allows the model to spend more time during learning analyzing high-quality texts, which dramatically accelerates convergence and improves the ability to solve complex tasks.

The direct application of the LLM-as-a-Judge concept is best illustrated in Microsoft's research titled *"Textbooks Are All You Need"* [11], which resulted in the **Phi** series of models. The essence of this approach is radical selection: instead of training the model on the entire internet filled with noise, it is trained exclusively on data with the structure and quality comparable to school textbooks. Results showed that a model trained on just **1%** of the originally planned data, but of extremely high semantic quality, can outperform models that are 50 times larger and trained on unprocessed internet noise.

7. Evolution Toward Synthetic Preprocessing: Knowledge Generation and Distillation

As the large language model industry faces a phenomenon known as the "**Data Wall**" — the point at which all high-quality human-generated content available on the internet has been exhausted — the focus of preprocessing is shifting radically. Instead of relying on raw, often contaminated data, modern pipelines use the most advanced models for actively creating and restructuring training sets.

7.1. Data Distillation and Teacher-Student Dynamics

Data distillation represents the process in which extremely large and powerful models (so-called Teacher models, e.g., GPT-5 or equivalent 2026 systems) generate high-quality answers, explanations, and problem solutions that are then used to train smaller, more efficient models (Student models). This technique addresses two key preprocessing problems:

- **Format consistency:** Synthetic data is always perfectly formatted, free of HTML noise or OCR errors.
- **Information density:** While average web text contains much filler, distilled data is designed so that every token carries the maximum amount of logical information.

7.2. Self-Correction and Automated Reasoning

One of the most advanced forms of preprocessing today is data generation through a self-correction process. In this scenario, the model generates an initial set of data (e.g., solutions to mathematics problems), and then, through special logic verification algorithms (such as Chain-of-Thought verification), identifies errors in its own reasoning. The final dataset that enters the training corpus contains only verified reasoning paths. In this way, preprocessing is no longer merely text filtering, but the filtering of logical processes. The result is a model that learns not only what the correct answer is, but also how to arrive at it without logical errors.

7.3. Curriculum Learning

Curriculum Learning represents a pedagogical approach in which data is not served to the model randomly, but in a strictly defined order of increasing difficulty. This process unfolds in real time during the preprocessing phase:

1. **Initial phase:** The model is exposed to simple, synthetically generated sentences and basic concepts ("elementary school" level).
2. **Intermediate phase:** Knowledge assessment algorithms identify that the model has mastered the basics, and the preprocessing system then unlocks more complex semantic structures and specialized literature.
3. **Advanced phase:** The model is confronted with the most difficult logical puzzles and specialized domains (coding, quantum physics).

This approach drastically reduces the time required for training, as the model does not waste computational resources on attempting to understand complex concepts before it has mastered the fundamentals.

7.4. Risk of Model Collapse

A key challenge of this state-of-the-art preprocessing approach is ***Model Collapse***. This is the phenomenon where a model, learning exclusively on synthetic data (generated by previous models), gradually loses contact with the real world and forgets rare but true information found only in human speech. For this reason, modern preprocessing in 2026 involves careful balancing: maintaining a ratio of approximately **20–30% authentic human data** against a synthetically generated core, in order to preserve the creativity and breadth of knowledge of the model.

8. Conclusion

The journey from raw, unprocessed internet noise to highly refined, semantically rich corpora represents the most significant paradigm shift in artificial intelligence development in recent years. The *Garbage In, Garbage Out* principle has evolved from a simple technical warning into the central strategic guideline of the industry: while early models relied on quantity and sheer statistical force, the modern AI revolution of 2026 is founded on the selection and intelligent synthesis of information. Through the analysis of algorithms such as **MinHash** and **LSH**, it has been demonstrated how the problem of redundancy at the scale of trillions of tokens is resolved, while the introduction of semantic models and the **LLM-as-a-Judge** methodology transforms the filtering process from passive cleaning into an active pursuit of textbook-level quality.

Although the transition to synthetic data, knowledge distillation, and **Curriculum Learning** leads to more efficient and powerful systems, the phenomenon of **Model Collapse** serves as a constant reminder that authentic human expression remains an irreplaceable foundation and ethical compass in the digital world of data. Ultimately, the success of future generations of large language models will not be measured solely by the number of parameters or kilowatt-hours consumed, but by the precision of curation that enables a machine to distill from an immense sea of information the essential human knowledge, logic, and values.

References

- [1] <https://machinelearningmastery.com/building-a-decoder-only-transformer-model-for-text-generation/>
- [2] <https://livebook.manning.com/wiki/categories/llm/causal+attention>
- [3] <https://arxiv.org/abs/2005.14165>
- [4] <https://arxiv.org/abs/2307.09288>
- [5] <https://arxiv.org/abs/2204.02311>
- [6] <https://arxiv.org/html/2503.10079v1>
- [7] WanJuan-CC: A Safe and High-Quality Open-sourced English Webtext Dataset
- [8] <https://openreview.net/forum?id=ak0vIKTkpx>
- [9] <https://tdan.com/data-speaks-for-itself-the-shift-from-syntactic-to-semantic-data-curation/31967>
- [10] <https://arxiv.org/abs/2306.08020>
- [11] <https://arxiv.org/abs/2306.11644>
- [12] <https://arxiv.org/abs/2410.04055>
- [13] <https://arxiv.org/abs/2406.04284>
- [14] <https://arxiv.org/abs/2101.10382>